

Preguntas – Introducción

1. Introducción al Desarrollo Web en Entorno Servidor

1.1 El Desarrollo Web Actual

- ¿Cuáles son los objetivos del despliegue?

a. Escalabilidad, accesibilidad, seguridad, estabilidad - **correcta**, justificación: Son objetivos fundamentales del despliegue para garantizar que la aplicación sea accesible, segura, estable y pueda crecer.

b. Accesibilidad, modularidad, horizontalidad, verticalidad - incorrecta, justificación: Son conceptos relacionados con el desarrollo y la escalabilidad, pero no representan los objetivos generales del despliegue.

c. Herencia, encapsulación, polimorfismo, jerarquía - incorrecta, justificación: Son conceptos propios de la programación orientada a objetos.

d. Gestionar dominios, gestionar dns, interactividad - incorrecta, justificación: Son tareas o elementos relacionados con una aplicación web, pero no objetivos generales del despliegue.

1.2 Modelos de Ejecución de Código en el Servidor y en el Cliente

- ¿Cuál de las siguientes opciones permite mejorar la experiencia de usuario, pero no significa una mejora en la seguridad?

a. Realizar Validaciones tanto en el back como en el front - incorrecta, justificación: Las validaciones en backend son necesarias para controlar y validar los datos de forma segura.

b. Las validaciones en el backend - incorrecta, justificación: Las validaciones en backend son una medida importante de seguridad.

c. Usar un framework de front como Angular, React o Vue - incorrecta, justificación: Un framework frontend mejora la estructura y experiencia de usuario, pero la pregunta busca específicamente una opción relacionada con validaciones.

d. Realizar validaciones en el cliente/front (navegador) - **correcta**, justificación: Las validaciones en el navegador mejoran la experiencia del usuario al proporcionar respuestas inmediatas, pero pueden ser manipuladas y por sí solas no aportan seguridad.

2. Componentes de una Web: Front-end, Back-end, Página Web vs. Aplicación Web

2.1 Front-end y Back-end: La División y la Universalidad del Back-end

- ¿Por qué se dice el backend debe ser universal o agnóstico?

a. Porque se desarrolla en un lenguaje de programación específico - incorrecta, justificación: Utilizar un lenguaje concreto no hace que el backend sea universal o agnóstico.

b. Porque el backend debe poder ser consumido por cualquier tipo de cliente, independientemente de las tecnologías con las que esté construido o desplegado. - **correcta**, justificación: El backend debe proporcionar servicios que puedan ser utilizados por diferentes tipos de clientes y tecnologías.

c. Porque es una tendencia - incorrecta, justificación: No es una cuestión de tendencia, sino de interoperabilidad y separación entre cliente y servidor.

d. Porque si no fuera por el backend no existiría el desarrollo web - incorrecta, justificación: El desarrollo web puede realizarse mediante diferentes arquitecturas y tecnologías.

2.2 Página Web vs. Aplicación Web: Diferencias y Tecnologías Clave

- ¿Qué tecnologías se usan en el backend?

a. Lenguajes de servidor como: PHP, Python, Ruby, Java, C# - **correcta**, justificación:

Son lenguajes que pueden ejecutarse en el servidor para implementar la lógica del backend.

b. Javascript, html y css - incorrecta, justificación: HTML y CSS se utilizan principalmente en el frontend; JavaScript puede utilizarse tanto en frontend como en backend.

c. Motores gráficos como Unity - incorrecta, justificación: Unity es principalmente un motor para videojuegos y aplicaciones interactivas, no una tecnología backend habitual.

d. Patrones de diseño como: builder, abstract factory, strategy etc. - incorrecta, justificación: Son patrones de diseño, no tecnologías o lenguajes de backend.

3. Arquitecturas web

3.1 Aspectos Generales y Evolución de las Arquitecturas Web

- ¿Cuáles son las ventajas de la arquitectura cliente-servidor?

a. Congestión del tráfico - incorrecta, justificación: La congestión del tráfico es un posible problema de una red, no una ventaja de esta arquitectura.

b. Requiere software y hardware específico - incorrecta, justificación: Es una característica o posible inconveniente, no una ventaja.

c. Velocidad a la hora de usar el software - incorrecta, justificación: La velocidad no es una ventaja inherente a la arquitectura cliente-servidor.

d. Centralización del control, escalabilidad, portabilidad, fácil mantenimiento - **correcta**, justificación: La separación entre cliente y servidor facilita la administración, el mantenimiento y la posibilidad de escalar el sistema.

3.2 Modelos de Arquitectura Software: Monolítica, de Capas, Microservicios y Serverless

- ¿Cuál es la diferencia entre una arquitectura de software monolítica y de capas respectivamente?

a. En la primera los componentes del sistema se comunican mediante eventos, utilizando suscripciones a estos eventos para reaccionar en consecuencia y en la otra el proveedor en la nube gestiona toda la infraestructura del servidor - incorrecta, justificación: La primera describe una arquitectura orientada a eventos y la segunda una arquitectura serverless, no monolítica y de capas.

b. En la primera todos los componentes de una aplicación se agrupan en un solo bloque, compartiendo recursos y dificultando muchas veces la escalabilidad, en la segunda la aplicación se dividen en diferentes módulos lógicos, donde cada uno tiene una responsabilidad específica, permite cambios en un módulo sin afectar los demás - **correcta**, justificación: Una arquitectura monolítica concentra los componentes en un único bloque, mientras que una arquitectura de capas separa las responsabilidades en diferentes módulos.

c. La primera se basa en tokens de acceso y la segunda se basa en el almacenamiento en local storage - incorrecta, justificación: Estos conceptos están relacionados con autenticación y almacenamiento, no con arquitecturas de software.

d. La primera pretende desarrollar soluciones basadas en la web y la segunda en desktop - incorrecta, justificación: Tanto las arquitecturas monolíticas como las de capas pueden utilizarse en aplicaciones web y de escritorio.

4. El Protocolo HTTP y HTTPS

4.1 Características y Ventajas del Protocolo HTTP

- ¿Por qué significa “Stateless”?

a. Significa que el servidor necesita un estado funcional que le permita ser operativo para responder peticiones http - incorrecta, justificación: Stateless significa precisamente que HTTP no mantiene el estado de una petición entre peticiones.

b. Significa que el servidor no necesita recordar quién eres, porque le estas pidiendo información. Sencillamente en cada petición debes enviarle una cookie/token valido que permita comprobar si puedes o no realizar dicha petición - **correcta**, justificación: HTTP no mantiene por defecto información de estado entre peticiones, por lo que la información necesaria debe enviarse en cada petición.

c. Significa que el estado de la petición http genera una sobrecarga en el rendimiento del servidor, por lo que se hace sin estado - incorrecta, justificación: Aunque no mantener estado puede tener ventajas de escalabilidad, esa no es la definición de Stateless.

d. Significa que los estados de cada petición llena la base de datos, por lo que se prefiere no almacenar estados - incorrecta, justificación: Stateless no significa que los estados se almacenen necesariamente en una base de datos.

4.2 Formato de Peticiones y Respuestas HTTP

- ¿Qué estructura tiene la primera línea de una petición http en orden?

a. Método, ruta del recurso solicitado, versión del protocolo - **correcta**, justificación: La línea inicial de una petición HTTP contiene el método, la ruta o recurso solicitado y la versión del protocolo.

b. Método, host, versión del protocolo - incorrecta, justificación: Host es una cabecera HTTP y no forma parte de la línea inicial.

c. Host, Método, ruta del recurso solicitado - incorrecta, justificación: El Host pertenece a las cabeceras y además el orden indicado no es el correcto.

d. Content-Type, Método, User-Agent, ruta del recurso solicitado - incorrecta, justificación: Content-Type y User-Agent son cabeceras y no forman parte de la primera línea de la petición.

4.3 Cabeceras HTTP

- Dos cabeceras de petición y dos de respuesta comunes

a. Accept, Content-Type, Cache-Control, Server - **correcta**, justificación: Accept y Content-Type son habituales en peticiones, mientras que Server y Cache-Control son habituales en respuestas.

b. Content-Type, Content-Language, Content-Length, Cache-Control - incorrecta, justificación: Son cabeceras que pueden aparecer principalmente en respuestas y no representan claramente dos de petición y dos de respuesta.

c. Accept, Accept-Language, Host, Content-Type, Content-Length - incorrecta, justificación: La opción contiene principalmente cabeceras de petición y además incluye cinco elementos en lugar de cuatro.

d. User-Agent, Server, Cache-Control, Content-Length - incorrecta, justificación: Server y Cache-Control son principalmente cabeceras de respuesta, por lo que no representa correctamente dos de cada tipo.

4.4 Métodos/Verbos HTTP (GET, POST, PUT, DELETE, HEAD)

- ¿Para que se usan los métodos http PUT y HEAD respectivamente?

- a. El primero para actualizar o reemplazar completamente un recurso y el segundo para solicitar las mismas cabeceras de respuesta que un método GET pero sin el cuerpo - **correcta**, justificación: PUT permite crear o reemplazar un recurso y HEAD solicita una respuesta equivalente a GET pero sin el cuerpo.
- b. El primero para realizar PUT al servidor y el segundo para traer la última respuesta http que generó el servidor - incorrecta, justificación: HEAD no recupera la última respuesta del servidor, sino que solicita las cabeceras correspondientes a un GET.
- c. El primero actualiza parcialmente un recurso y el segundo elimina el recurso - incorrecta, justificación: La actualización parcial se realiza normalmente mediante PATCH y HEAD no elimina recursos.
- d. El primero actualizar un recurso sin necesidad de autenticarse y el segundo lo mismo pero es necesario autenticarse antes - incorrecta, justificación: PUT y HEAD no determinan por sí mismos si es necesaria autenticación.

4.5 Códigos de Estado HTTP

- Los códigos de estado http 1xx, 2xx, 3xx, 4xx, 5xx respectivamente son respuestas:
 - a. Gráficas, de protección, de riesgo, ilustrativas - incorrecta, justificación: Estas categorías no corresponden a la clasificación de códigos HTTP.
 - b. de Redirección, error del cliente, error del servidor, informativas - incorrecta, justificación: Las categorías están incompletas y además aparecen en un orden incorrecto.
 - c. Informativas, de éxito, de redirección, error del cliente, error del servidor - **correcta**, justificación: Es la clasificación estándar de los códigos HTTP según su primera cifra.
 - d. De éxito, de redirección, informativas, de falla masiva, de protección - incorrecta, justificación: Las categorías no corresponden a la clasificación estándar de HTTP.

4.6 El Protocolo HTTPS (SSL/TLS y Certificados Digitales)

- ¿Qué hace seguro el protocolo HTTPS?
 - a. El cifrado de la información que viaja entre cliente y servidor, gracias a el uso de la llave pública del servidor integrad en un certificado digital emitido por una ca autorizada - **correcta**, justificación: HTTPS utiliza TLS para cifrar la comunicación y los certificados permiten autenticar al servidor.
 - b. El hecho de que sea un standard - incorrecta, justificación: Ser un estándar no implica por sí mismo que una comunicación sea segura.
 - c. El uso de servidores linux para el despliegue del backend - incorrecta, justificación: HTTPS puede utilizarse independientemente del sistema operativo del servidor.
 - d. El cifrado criptográfico que se guarda en la base de datos del servidor - incorrecta, justificación: HTTPS protege la comunicación entre cliente y servidor, no el almacenamiento de la base de datos.

5. Servicio Web y Comunicación con APIs

5.1 ¿Qué es un Servicio Web y Qué Aporta?

- ¿Qué es API y que permite?
 - a. Application Programming Interface y permite a otra aplicación comunicarse remotamente para acceder a su servicio - **correcta**, justificación: API significa Application Programming Interface y permite que diferentes aplicaciones se comuniquen y utilicen

funcionalidades o datos.

b. Application Programming Interface y permite almacenar datos de forma exitosa en un fichero - incorrecta, justificación: Una API no se define como un mecanismo específico para almacenar datos en ficheros.

c. Application Parameter Interface y permite parametrizar operaciones - incorrecta, justificación: API significa Application Programming Interface, no Application Parameter Interface.

d. Application Permissions Interpreter y permite interpretar los permisos que tiene o no un usuario - incorrecta, justificación: Ese no es el significado de las siglas API.

5.2 Diferencia entre un Servicio Web y una Página Web Dinámica

- ¿Cual es la diferencia entre un servicio web y una página web dinámica?

a. La página web está diseñada para generar contenido html u otros formatos que pueda interpretar el navegador, mientras que el servicio web expone datos y funcionalidades a los cuales se puede acceder a través de endpoints - **correcta**, justificación: Una página web está orientada a presentar contenido al usuario, mientras que un servicio web expone datos o funcionalidades para ser consumidos por otros clientes.

b. La página web es una landing page y el servicio web es el backend - incorrecta, justificación: Una página web no tiene por qué ser una landing page y un servicio web no es necesariamente todo el backend.

c. La página web es el cliente y el servicio web es el servidor - incorrecta, justificación: Una página web puede actuar como cliente de un servicio web, pero esta no es la diferencia principal.

d. El servidor guarda datos mientras que la pagina web los expone - incorrecta, justificación: Tanto una aplicación web como un servicio web pueden trabajar con datos; la diferencia principal está en cómo exponen y consumen la información.

5.3 Protocolos de Comunicación de API

- Tres protocolos de comunicación:

a. GRPC, WebSocket, rest - **correcta**, justificación: gRPC, WebSocket y REST son tecnologías utilizadas para establecer comunicación entre aplicaciones y servicios.

b. Network, wifi, bluetooth - incorrecta, justificación: Son conceptos o tecnologías de comunicación de red, pero no son protocolos de API equivalentes a REST, gRPC o WebSocket.

c. Smtip, redmark, wifi - incorrecta, justificación: SMTP es un protocolo de correo y los demás elementos no forman un conjunto de protocolos de API.

d. Rust, Oracle, .NET - incorrecta, justificación: Son un lenguaje y diferentes tecnologías/plataformas, no protocolos de comunicación de API.

5.4 JWT (JSON Web Token) - Concepto de Autenticación y Autorización

- ¿Cuales son las 3 partes del JWT?

a. Header, Payload, Signature - **correcta**, justificación: Un JWT está compuesto por Header, Payload y Signature.

b. Encabezado, pie de página, firma - incorrecta, justificación: Un JWT no tiene una parte denominada pie de página; utiliza Header, Payload y Signature.

c. Petición, respuesta, Content-Type - incorrecta, justificación: Son elementos relacionados con HTTP, no con las tres partes de un JWT.

d. Token, Cookie, local storage - incorrecta, justificación: Son mecanismos relacionados con el almacenamiento o transporte de información, no las partes internas de un JWT.

5.5 Ejemplo de arquitectura Netflix

- ¿Qué arquitectura usa Netflix?
 - a. Monolítica - incorrecta, justificación: Netflix evolucionó desde una arquitectura monolítica hacia una arquitectura basada principalmente en microservicios.
 - b. Microservicios - **correcta**, justificación: Netflix es uno de los ejemplos más conocidos de utilización de una arquitectura basada en microservicios.
 - c. Capas - incorrecta, justificación: Aunque pueden existir componentes organizados por capas, no es la arquitectura característica por la que destaca Netflix.
 - d. SOA - incorrecta, justificación: Netflix se caracteriza principalmente por el uso de microservicios, aunque existen similitudes conceptuales con SOA.

6. Funcionamiento de una Web Dinámica y Generación de Contenido

6.1 Páginas estáticas vs. Páginas dinámicas

- ¿Diferencia entre webs estáticas y dinámicas respectivamente?
 - a. Una tiene el mismo contenido para todos los usuarios y la otro lo genera dinámicamente en el servidor en el momento que el usuario las solicita - **correcta**, justificación: Una web estática sirve contenido previamente generado, mientras que una web dinámica puede generar contenido según la petición del usuario.
 - b. Una tiene contenido estático y la otra dinámico - incorrecta, justificación: Aunque la afirmación es cierta, es demasiado general y no explica la diferencia de funcionamiento entre ambas.
 - c. Una se muestra en el navegador y la otra no - incorrecta, justificación: Tanto las páginas estáticas como las dinámicas pueden mostrarse en el navegador.
 - d. Una permite realizar peticiones http y la otra no - incorrecta, justificación: Ambas pueden utilizar peticiones HTTP.

6.2 Cómo Funciona una Página Web Dinámica

- ¿Cual es el primer y ultimo paso en el funcionamiento de una web dinámica?
 - a. Envío de la página web al navegador, solicitud de la página web al servidor - incorrecta, justificación: El orden es incorrecto porque primero se realiza la solicitud y después se envía la respuesta.
 - b. Solicitud de la página web al servidor, envío de la página web al navegador - **correcta**, justificación: El navegador solicita el recurso al servidor y posteriormente recibe la página como respuesta.
 - c. Envío de formulario, respuesta http - incorrecta, justificación: No todas las peticiones de una web dinámica implican el envío de un formulario.
 - d. Validación de datos, Envío de la información web - incorrecta, justificación: Son acciones que pueden producirse, pero no representan necesariamente el primer y último paso del proceso.

6.3 Tecnologías de Generación de Páginas Web Dinámicas y Ejemplos "Hola Mundo"

- Tecnologías para generar webs dinámicas:

- a. PHP/JAVA - **correcta**, justificación: PHP y Java pueden utilizarse en el servidor para generar contenido web dinámico.
- b. JavaScript, IntelliJ - incorrecta, justificación: JavaScript sí puede utilizarse en backend, pero IntelliJ es un entorno de desarrollo, no una tecnología de generación web.
- c. VisualStudio, VisualStudioCode - incorrecta, justificación: Son entornos de desarrollo y editores, no lenguajes o tecnologías de generación de páginas dinámicas.
- d. Ninguna de las anteriores - incorrecta, justificación: PHP y Java sí son tecnologías válidas para generar contenido web dinámico.

7. Lenguajes y Frameworks de Programación en Entorno Servidor

7.1 Tipos de Ejecución de Lenguajes del Lado del Servidor

- ¿En que categorías se dividen los lenguajes de programación de lado servidor?
 - a. Lenguajes de marcas, Lenguajes técnicos - incorrecta, justificación: No es una clasificación habitual de los lenguajes de programación del lado servidor.
 - b. Lenguajes criptográficos, lenguajes no-criptográficos - incorrecta, justificación: La clasificación no se realiza según sean criptográficos o no.
 - c. Lenguajes de scripting, Lenguajes compilados, Lenguajes compilados a código intermedio - **correcta**, justificación: Es una clasificación basada en la forma en que se ejecuta y procesa el código.
 - d. Ninguna de las anteriores - incorrecta, justificación: La opción C proporciona una clasificación válida.

7.2 Plataformas Web Libres y Propietarias

- ¿Cuales son algunas de las combinaciones populares o stack para desarrollo de software?
 - a. LAMP, MEAN / MERN - **correcta**, justificación: LAMP, MEAN y MERN son stacks conocidos y utilizados para el desarrollo de aplicaciones web.
 - b. AAR, LOE, VI - incorrecta, justificación: No son stacks tecnológicos web reconocidos.
 - c. VS code, Apache, Linux - incorrecta, justificación: Son herramientas y tecnologías que pueden utilizarse conjuntamente, pero no forman uno de los stacks indicados.
 - d. Todas las anteriores - incorrecta, justificación: No puede ser correcta porque la opción B no corresponde a stacks reconocidos.

7.3 Tecnologías para el Desarrollo de Servicios

- Tecnologías para el desarrollo de servicios:
 - a. Java(Spring boot), C#(Asp.net core), PHP(Laravel) - **correcta**, justificación: Spring Boot, ASP.NET Core y Laravel son frameworks utilizados para desarrollar servicios y aplicaciones web.
 - b. Html, css - incorrecta, justificación: HTML y CSS se utilizan principalmente para estructurar y dar estilo al frontend.
 - c. Js - incorrecta, justificación: JavaScript puede utilizarse en backend, pero la opción no identifica una tecnología o framework concreto de servicios.
 - d. Office word, excel - incorrecta, justificación: Word y Excel son aplicaciones ofimáticas y no tecnologías para desarrollar servicios web.

7.4 Integración del Código con Lenguajes de Marcas

- ¿En mvc como se renderizan las páginas?

a. El lenguaje de servidor java, php, c# se incrusta en el html a través de anotaciones para posterior mente ser renderizadas. - **correcta**, justificación: En MVC se utilizan plantillas y tecnologías del lado servidor para combinar los datos con el HTML y generar la respuesta que recibe el navegador.

b. Se usa solamente js a la hora de renderizar las web - incorrecta, justificación: MVC no requiere utilizar únicamente JavaScript para renderizar las páginas.

c. Se renderizan gracias a el uso de librerías externas - incorrecta, justificación: Las librerías pueden utilizarse, pero no son por sí mismas la característica que define el renderizado en MVC.

d. Se renderizan dependiendo del lenguaje en el que este desarrollada la aplicación - incorrecta, justificación: El lenguaje utilizado influye en la implementación, pero no explica por sí solo cómo funciona el renderizado en MVC.

8. Funcionamiento y Configuración de Servidores Web y de Aplicaciones

8.1 Servidores Web: Apache y Nginx

- ¿Qué es un servidor web?

a. Es el “portero” de nuestra aplicación web. Escucha el puerto 80 o 443, sirve ficheros estáticos o dinámicos - **correcta**, justificación: Un servidor web recibe peticiones HTTP/HTTPS y puede servir recursos estáticos o gestionar peticiones hacia contenido dinámico.

b. Es una configuración de linux para optimizar recursos - incorrecta, justificación: Un servidor web es un software que atiende peticiones, no una configuración específica de Linux.

c. Es una dependencia adicional que se añade en el proyecto de software - incorrecta, justificación: Un servidor web es un componente independiente que proporciona servicios HTTP, no simplemente una dependencia del proyecto.

d. Es una guía para exponer las funcionalidades de tu aplicación web - incorrecta, justificación: Un servidor web es software que recibe y procesa peticiones, no una guía.

8.2 Servidores de Aplicaciones: Tomcat

- ¿Qué es Apache Tomcat?

a. Una distro de linux - incorrecta, justificación: Tomcat es un servidor de aplicaciones y no un sistema operativo.

b. Un protocolo de transferencia de información - incorrecta, justificación: Tomcat no es un protocolo de comunicación.

c. Servidor de aplicaciones y contenedor de servlets y jsps por excelencia - **correcta**, justificación: Apache Tomcat es un servidor y contenedor utilizado para ejecutar aplicaciones Java basadas en servlets y JSP.

d. Un motor de bases de datos - incorrecta, justificación: Tomcat no es un sistema gestor de bases de datos.

8.3 Gestores de Bases de Datos

- Motores de bases de datos:

- a. Apache Tomcat - incorrecta, justificación: Tomcat es un servidor de aplicaciones, no un motor de bases de datos.
- b. Virtual Box - incorrecta, justificación: VirtualBox es un software de virtualización.
- c. MySql - **correcta**, justificación: MySQL es un sistema gestor de bases de datos relacional.
- d. PostresSql - incorrecta, justificación: PostgreSQL también es realmente un sistema gestor de bases de datos, por lo que esta pregunta contiene técnicamente dos respuestas correctas. Si el examen exige una única respuesta, la opción esperada parece ser MySQL.

9. Despliegue de Aplicaciones Web

9.1 Concepto de Despliegue

- ¿Qué es el despliegue de una aplicación?
 - a. Es una metodología de desarrollo de software - incorrecta, justificación: El despliegue es un proceso para poner una aplicación en un entorno donde pueda ejecutarse, no una metodología de desarrollo.
 - b. Es el proceso de llevar tu código desde tu ordenador ("localhost") hasta un servidor accesible por los usuarios ("producción"). - **correcta**, justificación: El despliegue consiste en poner la aplicación en un entorno accesible para los usuarios.
 - c. Es el stack que se utiliza para desarrollar la aplicación - incorrecta, justificación: El stack es el conjunto de tecnologías utilizadas para desarrollar la aplicación.
 - d. Es el backup periódico que se hace de la información - incorrecta, justificación: Un backup es una copia de seguridad y no un despliegue.

9.2 Escalabilidad (Vertical y Horizontal, Clusters y Balanceadores de Carga)

- Diferencia entre escalabilidad vertical y horizontal:
 - a. En la primera se contrata mas personal para desarrollo y en la segunda se aumenta la productividad - incorrecta, justificación: La escalabilidad informática se refiere a recursos tecnológicos, no al número de trabajadores.
 - b. En la primera se cambia de motor de base de datos y en la segunda se cambia de lenguaje de programación - incorrecta, justificación: Cambiar tecnologías no define la escalabilidad vertical u horizontal.
 - c. En la primera se actualiza la seguridad del servidor y en la segunda actualizan los paquetes - incorrecta, justificación: Son tareas de mantenimiento y seguridad, no tipos de escalabilidad.
 - d. En la primera se le asigna mas hardware al servidor vps y en la segunda se configuran más servidores generando alta disponibilidad y redistribución de la carga - **correcta**, justificación: La escalabilidad vertical aumenta los recursos de un servidor, mientras que la horizontal añade más servidores para repartir la carga.

9.3 Despliegue en Contenedores (Docker, Kubernetes)

- ¿Qué función cumple kubernestes (k8s)?
 - a. Orquestación masiva. Gestiona miles de contenedores, autoescalado, recuperación de fallos, etc. - **correcta**, justificación: Kubernetes es una plataforma de orquestación que permite gestionar, escalar y recuperar aplicaciones basadas en contenedores.
 - b. Proveer seguridad al servidor - incorrecta, justificación: Kubernetes dispone de mecanismos de seguridad, pero su función principal es la orquestación de contenedores.

- c. Configurar variables de entorno - incorrecta, justificación: Kubernetes puede gestionar configuraciones y variables, pero esa no es su función principal.
- d. Todas las anteriores - incorrecta, justificación: La función principal de Kubernetes es la orquestación; las otras opciones describen capacidades secundarias y no su función principal.

9.4 Despliegue en la Nube (AWS, Google Cloud, Azure)

- ¿Qué significa IaaS, PaaS, SaaS respectivamente:
 - a. Information as a Service, Product as a Service, Security as a Service - incorrecta, justificación: Estas no son las definiciones correctas de IaaS, PaaS y SaaS.
 - b. Internet as a Service, Programming as a Service, System as a Service - incorrecta, justificación: Estas expansiones no corresponden a las siglas.
 - c. A y B son correctas - incorrecta, justificación: Las opciones A y B son incorrectas, por lo que esta opción tampoco puede ser correcta.
 - d. Infrastructure as a Service, Platform as a Service, Software as a Service - **correcta**, justificación: Son las definiciones estándar de los tres principales modelos de servicio en la nube.

9.6 Introducción a la Integración Continua / Despliegue Continuo (CI/CD)

- ¿Qué significa CI/CD?
 - a. Continuous Interfaces / Continuous Delivery - incorrecta, justificación: CI no significa Continuous Interfaces.
 - b. Continuous Integration / Continuous Delivery - **correcta**, justificación: CI significa Continuous Integration y CD puede significar Continuous Delivery.
 - c. A y B - incorrecta, justificación: La opción A es incorrecta, por lo que no pueden ser ambas correctas.
 - d. Solo B - incorrecta, justificación: Esta opción indica que únicamente B es correcta, pero no es una definición de CI/CD; la respuesta que proporciona directamente el significado es B.

10. Conceptos Clave de Seguridad y Monitorización

10.1 Seguridad en Servidores Web (Autenticación HTTP Basic/Digest, Control de Acceso)

- Diferencia entre autenticación y autorización:
 - a. La primera permite que el servidor sepa quién eres y la segunda que permisos tienes - **correcta**, justificación: La autenticación comprueba la identidad del usuario y la autorización determina qué permisos o acciones tiene.
 - b. La primera es jwt la segunda cookie de sesión - incorrecta, justificación: JWT y las cookies son mecanismos que pueden utilizarse en procesos de autenticación, pero no son la diferencia entre autenticación y autorización.
 - c. La primera es stateless la segunda tiene estado - incorrecta, justificación: Stateless y stateful describen cómo se gestiona el estado, no la diferencia entre autenticación y autorización.
 - d. Ninguna de las anteriores - incorrecta, justificación: La opción A define correctamente la diferencia entre autenticación y autorización.

10.2 Monitorización con Archivos de Registro (Logs CLF, Rotación de Logs)

- ¿Para que sirven los logs?

a. Sirven para llevar registro registro de las peticiones y procesos que se realizan en la aplicación, permitiendo analizar errores por ejemplo - **correcta**, justificación: Los logs registran eventos, peticiones y procesos de una aplicación y permiten detectar y analizar errores.

b. Sirven para comprobar las versiones de las dependencias del proyecto - incorrecta, justificación: Las versiones de las dependencias se gestionan normalmente mediante archivos de configuración y gestores de paquetes, no mediante logs.

c. Sirven para exponer funcionalidades de la aplicación - incorrecta, justificación: Los logs registran información sobre la aplicación, pero no sirven para exponer sus funcionalidades a los usuarios.

d. Todas las anteriores - incorrecta, justificación: Las opciones B y C son incorrectas, por lo que no pueden ser todas correctas.